

Plan de Implementacion

Sistema de Autenticacion 2FA

Fecha: 03/06/2026

Version: 1.0

Proyecto: API_2FA - FastAPI + MySQL

Documento generado a partir del analisis del codigo fuente y la documentacion del proyecto.

. Indice

1. Resumen Ejecutivo
2. Diagnostico del Proyecto Actual
3. Discrepancias: Documentacion vs Implementacion
4. Plan de Implementacion por Fases
 - Fase 1: Correccion de Bugs Criticos
 - Fase 2: Seguridad
 - Fase 3: Funcionalidad 2FA Completa
 - Fase 4: Frontend Completo
 - Fase 5: Arquitectura yCodigo
 - Fase 6: Testing y Documentacion
5. Roadmap y Estimacion Temporal
6. Criterios de Aceptacion

1. Resumen Ejecutivo

El presente plan de implementacion tiene como objetivo mejorar integralmente el sistema de autenticacion de dos factores (2FA) desarrollado en FastAPI con MySQL. El proyecto fue desarrollado en 3 sprints previos (Sprint I: Configuracion base, Sprint II: Generacion de OTP, Sprint III: Validacion de OTP) y actualmente presenta bugs criticos que inhabilitan funcionalidades completas, asi como aspectos de seguridad y experiencia de usuario que requieren atencion.

Este plan contempla 6 fases de implementacion que abordan desde la correccion de errores criticos hasta mejoras visuales y de testing. Se ha realizado un analisis exhaustivo del codigo fuente y se han identificado 18 hallazgos entre bugs, vulnerabilidades y funcionalidad incompleta, asi como discrepancias entre la documentacion contractual y la implementacion actual.

2. Diagnostico del Proyecto Actual

2.1 Bugs Criticos

#	Problema	Archivo	Impacto
1	Router sobrescrito: router = APIRouter	Routers/auth.py:326	ALTO - 0 rutas
	todas las rutas admin definidas previam		funcionan
2	Router telefono importado pero nunca r	main.py:17	ALTO - /telefonos
	Falta app.include_router(telefono.router)		no responde

2.2 Problemas de Seguridad

#	Problema	Localizacion	Severidad
3	Contraseñas almacenadas y comparad	auth.py:22, authService.py	CRITICO
4	crear_usuario.py usa bcrypt pero login	crear_usuario.py vs auth.py	CRITICO
5	JWT secret con fallback hardcodeado	jwtUtil.py:7	ALTO
6	OTP impreso en consola, nunca enviado	auth.py:367	MEDIO
7	SHA256 para hash de OTP (deberia se	securityUtil.py:9	MEDIO
8	CORS permite todos los origenes (*)	main.py:15	MEDIO

2.3 Funcionalidad Incompleta

#	Problema	Archivo	Estado
9	Archivos de servicio/repositorio vacios	otpService.py, userRepo.py, otpUtil.py	VACIOS
10	No hay UI para verificar OTP	frontend/index.html	INCOMPLETO
11	No hay panel de administracion frontend	frontend/	FALTA
12	No hay UI de validacion de telefonos	frontend/	FALTA
13	.env solo tiene DATABASE_URL	.env	INCOMPLETO
14	API keys de reCAPTCHA hardcodeada	frontend/index.html:9,44	MALA PRACTICA
15	No existe requirements.txt	Raiz	FALTA

2.4 Problemas de Frontend/UX

- Solo existe un formulario basico de generacion de OTP
- No hay flujo completo: Login -> Captcha -> OTP -> Verify -> JWT
- Sin disenio responsive avanzado, ni temas
- Sin validacion visual de errores en frontend
- Sin loading states, animaciones, ni feedback visual para el usuario

3. Discrepancias: Documentacion vs Implementacion

Se analizaron los documentos "Sprints.pdf" y "Documentacion2FA.pdf" y se encontraron las siguientes discrepancias entre lo especificado y lo implementado:

3.1 Modelo de Datos (tb_users)

Campo (Documentacion)	Esperado	Real (Codigo)	Accion
two_factor_enabled	boolean	NO EXISTE	Agregar al modelo
two_factor_length	int	otp_length (int)	Renombrar/mapear
two_factor_method	int (1=SMS,2=Email,3=Ambos)	NO EXISTE	Agregar al modelo
two_factor_type	int (1=numerico,2=alfanumerico)	otp_type (str)	Estandarizar

3.2 Modelo de Datos (tb_two_factor_codes)

Campo (Documentacion)	Esperado	Real (Codigo)	Accion
code_destinator	string (email/tel)	NO EXISTE	Agregar al modelo
created_at	datetime	NO EXISTE	Agregar al modelo

3.3 Endpoint /auth/generate

Aspecto	Documentacion	Implementacion	Accion
Response structure	errorCode, status, errorMessage,	mensaje, destinator_hint	Estandarizar respuesta
	user_id, destinator_hint		
Status code exito	errorCode: 0, status: "2FA_REQUIRED"	HTTP 200 + "mensaje"	Corregir formato
Error codes	101 (credenciales), 102 (captcha),	HTTP 400, 401, 403	Migrar a codigos
	500 (interno)		estandarizados

3.4 Endpoint /auth/2fa/verify

Aspecto	Documentacion	Implementacion	Accion
Ruta exacta	/auth/2fa/verify	/auth/2fa/verify	OK (coincide)
Response exito	errorCode: 0, status: "SUCCESS",	Coincide parcialmente	Ajustar formato
	errorMessage: "OK", access_token		
Error codes	201 (incorrecto), 202 (expirado),	Coinciden	OK
	203 (intentos), 500 (interno)		

4. Plan de Implementacion por Fases

Fase 1: Correccion de Bugs Criticos

Objetivo: Restaurar la funcionalidad completa del sistema existente.

- **1.1** Refactorizar auth.py: eliminar la doble definicion de router, unificar todos los endpoints en un solo router, separar rutas admin a Routers/admin.py
- **1.2** Registrar telefono.router en main.py con app.include_router(telefono.router)
- **1.3** Crear archivo Routers/__init__.py para gestion de importaciones limpia
- **1.4** Probar manualmente cada endpoint existente y verificar respuestas HTTP correctas

Archivos: Routers/auth.py, main.py, Routers/telefono.py (verificar)

Fase 2: Seguridad

Objetivo: Eliminar vulnerabilidades criticas y proteger datos sensibles.

2.1 Hashing de Contraseñas

- Agregar passlib + bcrypt a requirements.txt
- Crear funcion hash_password(password) -> str y verify_password(plain, hashed) -> bool en securityUtil.py
- Modificar crear_usuario.py para usar la misma funcion hash centralizada
- Modificar endpoint /auth/generate para usar verify_password en lugar de comparacion directa
- Modificar endpoint /admin/usuarios (POST y PUT) para hashear contraseñas al crear/actualizar

2.2 JWT Seguro

- Eliminar fallback hardcodeado "tu_clave_secreta_por_defecto" de jwtUtil.py
- Agregar validacion: si no existe jwtSecret en .env, lanzar error al iniciar
- Agregar expiracion configurable via variable de entorno (jwtExpirationHours)
- Agregar validacion/decodificacion de tokens en jwtUtil.py (actualmente solo crea)

2.3 OTP Seguro

- Cambiar hash de OTP de SHA256 a bcrypt (uso de hash_code con bcrypt)
- Mantener proteccion contra timing attacks (hmac.compare_digest)

2.4 Configuracion de Entorno

- Agregar variables faltantes al .env: RECAPTCHA_SECRET_KEY, jwtSecret, jwtExpirationHours, otpExpiration, otpMaxAttempts
- Mover reCAPTCHA site key del HTML a variable de entorno, pasarla desde el backend
- Agregar validacion de variables requeridas al iniciar la aplicacion

2.5 Logging de Seguridad

- Implementar logging estructurado de intentos fallidos de autenticacion
- Registrar intentos de acceso a rutas admin sin permisos
- Registrar generacion y verificacion de OTP (sin incluir el codigo)

Archivos: Utils/securityUtil.py, Utils/jwtUtil.py, Services/authService.py, Routers/auth.py, crear_usuario.py, Config/db.py, .env

Fase 3: Funcionalidad 2FA Completa

Objetivo: Implementar el flujo completo de 2FA segun la documentacion, incluyendo envio real de codigos.

3.1 Alinear Modelo de Datos con Documentacion

- Agregar campos faltantes a Usuarios: `two_factor_enabled` (bool), `two_factor_method` (int)
- Agregar campos faltantes a TwoFactorCodes: `code_destinator` (str), `created_at` (datetime)
- Agregar campo `email_valido` y `telefono_valido` al modelo Usuarios

3.2 Implementar Envio de OTP

- Completar `otpService.py`: logica de generacion, expiracion, y seleccion de canal
- Implementar envio por email via SMTP (configurado via `.env`)
- Implementar envio por SMS (conector para API de SMS, configurable)
- Completar `otpUtil.py`: funciones de generacion con soporte para tipo y longitud

3.3 Estandarizar Respuestas de API

- Endpoint `/auth/generate`: cambiar respuesta a formato estandar (`errorCode`, `status`, `errorMessage`, `user_id`, `destinator_hint`)
- Endpoint `/auth/2fa/verify`: mantener formato estandar actual, verificar consistencia
- Mapear codigos de error segun documentacion: 0 (OK), 101 (credenciales), 102 (captcha), 201 (OTP incorrecto), 202 (expirado), 203 (intentos), 500 (interno)

3.4 Rate Limiting y Reintentos

- Implementar rate limiting en generacion de OTP (ej. max 3 solicitudes cada 5 minutos)
- Endpoint para reenviar OTP: `/auth/resent-otp`
- Bloquear temporalmente generacion de OTP tras exceder limite de reintentos

3.5 Completar Repositorios y Servicios

- Completar `userRepo.py`: funciones `get_user_by_username`, `create_user`, `update_user`, `delete_user`
- Completar `twoFactorRepo.py`: funciones `get_latest_otp`, `increment_attempts`, `mark_as_used`, `invalidate_old_codes`
- Completar `otpService.py`: `generate_otp`, `send_otp`, `verify_otp`, `resent_otp`
- Completar `authService.py`: flujo completo de autenticacion con 2FA

Archivos: `Models/models.py`, `Services/otpService.py`, `Services/authService.py`, `Repositories/userRepo.py`, `Repositories/twoFactorRepo.py`, `Routers/auth.py`, `Util`

Fase 4: Frontend Completo

Objetivo: Proporcionar una interfaz de usuario completa, moderna y funcional para todo el flujo 2FA.

4.1 Pagina de Login con reCAPTCHA

- Formulario de inicio de sesion con usuario y contraseña
- Integracion con reCAPTCHA v3 (invisible)
- Validacion de campos en tiempo real
- Manejo de errores con mensajes descriptivos

4.2 Pantalla de Verificacion OTP

- Input de codigo OTP con mascara por caracter
- Timer de expiracion visible (ej. 5:00 -> 0:00)
- Opcion de reenviar codigo (con cooldown)
- Indicador de intentos restantes
- Loading spinner durante verificacion

4.3 Dashboard de Usuario

- Pagina post-autenticacion mostrando datos del usuario
- Informacion del token JWT activo
- Opcion de cerrar sesion
- Historial de inicio de sesion (ultimos accesos)

4.4 Panel de Administracion

- CRUD de usuarios: listar, crear, editar, eliminar (con confirmacion)
- Gestion de tokens: generar, ligar a usuario, activar/desactivar
- Gestion de APIs: listar, crear, editar, eliminar, activar/desactivar
- Carga de CSV de numeraciones con preview y feedback
- Tabla de numeraciones locales con busqueda y filtros

4.5 Modulo de Validacion de Telefonos

- Formulario de consulta de numero telefonico
- Resultados con codigos de estado (01=exito API, 02=no valido, 03=cache/local)
- Historial de consultas realizadas
- Selector de API especifica o modo automatico

4.6 Diseño Visual

- Bootstrap 5.3 + tema personalizado (colores corporativos)
- Responsive design (mobile-first)
- Notificaciones toast para feedback de acciones
- Skeleton loading en componentes asincronos
- Transiciones y animaciones suaves
- Modo oscuro/claro

Archivos: *frontend/* (reescritura completa)

Fase 5: Arquitectura yCodigo

Objetivo: Mejorar la calidad, mantenibilidad y robustez del codigo existente.

5.1 Reestructuracion de Routers

- Separar auth.py en: Routers/auth.py (login, 2FA), Routers/admin.py (CRUDs, gestion)
- Mantener compatibilidad hacia atras donde sea posible
- Organizar imports de forma consistente

5.2 Manejo de Errores Global

- Crear middleware de manejo de excepciones global
- Estandarizar formato de respuesta de error en toda la API
- Agregar logging automatico de errores no controlados

5.3 Validacion de Datos

- Reforzar validacion Pydantic en todos los esquemas de entrada
- Agregar validacion de formato de telefono, email, contraseñas
- Sanitizar entradas para prevenir inyeccion

5.4 Configuracion y Dependencias

- Crear requirements.txt con todas las dependencias
- Centralizar configuracion en un archivo Config/settings.py usando Pydantic Settings
- Agregar validacion de configuracion al iniciar la aplicacion

5.5 Completar Archivos Vacios

- Completar otpService.py con funciones de generacion, expiracion y canal
- Completar userRepo.py con CRUD de usuarios
- Completar otpUtil.py con generacion de OTP (actualmente la logica esta en securityUtil)

Archivos: Múltiples (ver detalle en cada tarea)

Fase 6: Testing y Documentacion

Objetivo: Garantizar la calidad del software y facilitar su mantenimiento futuro.

6.1 Testing

- Configurar pytest con base de datos de prueba (SQLite en memoria)
- Tests unitarios para Utils: test_hash_password, test_verify_otp, test_generate_otp, test_jwt
- Tests unitarios para Services: test_auth_service, test_otp_service, test_recaptcha_service
- Tests de integracion para endpoints: POST /auth/generate, POST /auth/2fa/verify
- Tests de integracion para rutas admin: CRUD usuarios, tokens, APIs
- Tests de integracion para validacion de telefonos

6.2 Documentacion

- Actualizar README.md con instrucciones de instalacion, configuracion y ejecucion
- Agregar ejemplos de peticiones y respuestas de cada endpoint
- Documentar variables de entorno requeridas
- Documentar la estructura del proyecto
- Crear CHANGELOG.md con el registro de cambios de esta implementacion

6.3 Seed de Datos de Prueba

- Crear script seed.py que genere datos de prueba (usuarios, tokens, APIs, numeraciones)
- Incluir un usuario admin por defecto
- Incluir APIs de ejemplo (numeracion local, API externa simulada)

Archivos: tests/, README.md, CHANGELOG.md, seed.py

5. Roadmap y Estimacion Temporal

Estimacion basada en orden secuencial. Cada fase depende de la anterior.

Fase	Descripcion	Prioridad	Tiempo Est.
1	Correccion de Bugs Criticos	ALTA	2-3 horas
2	Seguridad	ALTA	4-6 horas
3	Funcionalidad 2FA Completa	ALTA	6-8 horas
4	Frontend Completo	MEDIA	8-12 horas
5	Arquitectura yCodigo	MEDIA	4-6 horas
6	Testing y Documentacion	MEDIA	4-6 horas

Tiempo Total Estimado: 28 - 41 horas

Nota: El tiempo estimado puede variar segun la complejidad de los issues encontrados durante la implementacion. Se recomienda realizar una revision intermedia al finalizar la Fase 3 para ajustar estimaciones de las fases restantes.

6. Criterios de Aceptacion Generales

6.1 Funcionalidad

- El sistema levanta sin errores con uvicorn
- El endpoint /auth/generate valida credenciales, captcha, genera OTP y responde en formato estandar
- El endpoint /auth/2fa/verify valida OTP, controla intentos, expiracion y genera JWT
- Las rutas admin (usuarios, tokens, APIs, CSV) responden correctamente
- Las rutas de validacion de telefonos (/telefonos) responden correctamente
- El frontend permite el flujo completo: Login -> Captcha -> OTP -> Verify -> Dashboard

6.2 Seguridad

- Las contraseñas se almacenan hasheadas con bcrypt
- No existe fallback hardcodeado de JWT secret
- Los OTPs se almacenan hasheados
- No se imprimen codigos OTP en consola/logs en produccion
- CORS configurado apropiadamente para el entorno
- Las variables de entorno sensibles no estan hardcodeadas

6.3 Calidad

- No hay archivos vacios ni imports no utilizados
- Las respuestas de API siguen el formato estandar definido en la documentacion
- Los tests unitarios pasan correctamente
- La aplicacion maneja errores con mensajes descriptivos y codigos apropiados

6.4 UX/Frontend

- El flujo de autenticacion 2FA es intuitivo y guiado
- El panel de administracion permite todas las operaciones CRUD
- La validacion de telefonos muestra resultados claros con los codigos de estado
- El diseno es responsive y funciona en dispositivos moviles